END-TO-END CI/CD PROJECT

GitHub → Jenkins → Docker → AWS EC2

Step-by-step setup, pipeline configuration, webhook, deployment, and troubleshooting

1. GitHub repository Application source code + Dockerfile + Jenkinsfile
2. GitHub webhook Sends a notification when code is pushed
3. Jenkins pipeline Checkout → build Docker image → deploy container
4. Docker on EC2 Runs the application in a container
5. Public website http://YOUR_EC2_PUBLIC_IP

Learning setup: Jenkins and Docker run on one EC2 instance.

What you will need

- An AWS account with permission to launch EC2 instances.
- A GitHub account and a repository (public is simplest for this project).
- A computer with a browser and, if using SSH from Windows, a terminal such as PowerShell.
- Basic comfort copying commands into an Ubuntu terminal.

Project choices used in this guide

- OS: Ubuntu Server 24.04 LTS, 64-bit x86.
- Instance: m7i-flex.large may be smoother for Jenkins + Docker -> (2 processor and min 4GB Ram). Confirm current regional pricing and your account's credits/Free Tier eligibility.
- Jenkins: Long-Term Support (LTS), installed from the official Jenkins apt repository.
- Sample app: a simple static HTML page served by Nginx in Docker. If your existing project is React or another framework, adapt its Dockerfile/build steps.
- Jenkins URL: http://EC2_PUBLIC_IP:8080 during learning; app URL: http://EC2_PUBLIC_IP.

Contents

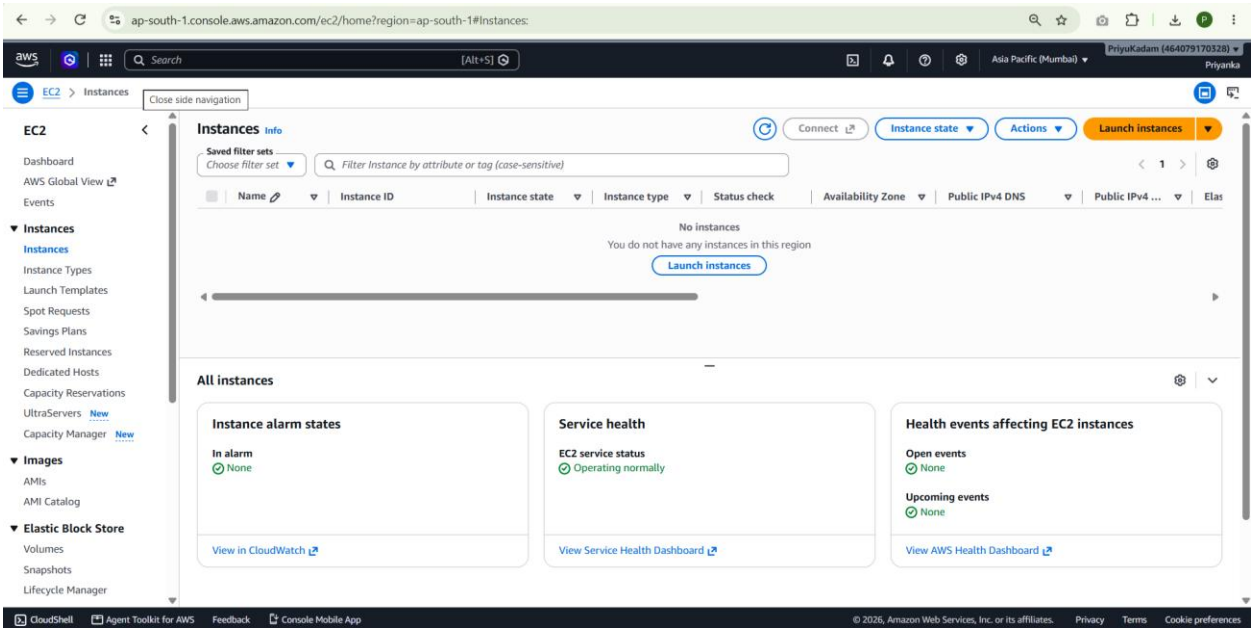
1. Launch the EC2 instance
2. Connect to Ubuntu
3. Install Java and Jenkins
4. Complete Jenkins first-time setup
5. Install Docker and allow Jenkins to run it
6. Create the GitHub project
7. Add the Dockerfile and Jenkinsfile
8. Create the Jenkins pipeline job
9. Configure the GitHub webhook
10. Run and verify the deployment
11. Troubleshooting
12. Security and cost cleanup
Official documentation

1. Launch the EC2 instance

Open the AWS console and create the Linux virtual machine that will host Jenkins and the application.

1.1 Open EC2

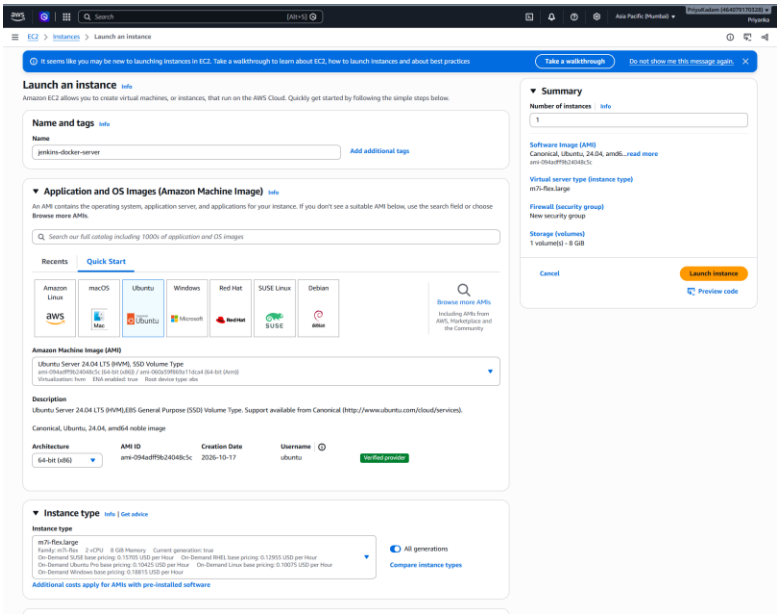
1. Sign in to AWS Console and open EC2: https://console.aws.amazon.com/ec2/
2. Choose the AWS Region where you want to create the instance. Keep the region noted.
3. Click Launch instance.



1.2 Configure the instance

- Name: jenkins-docker-server
- Application and OS Image (AMI): Ubuntu Server 24.04 LTS, 64-bit (x86).
- Instance type: m7i-flex.large may be smoother for Jenkins + Docker -> (2 processor and min 4GB Ram).

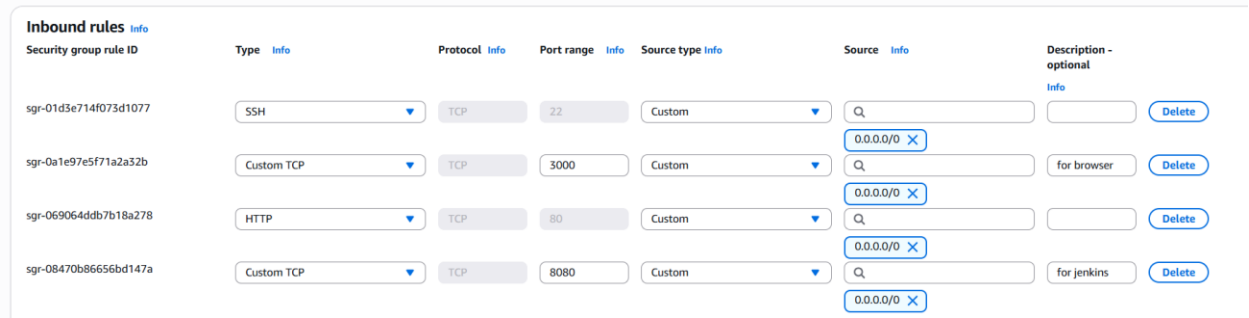
- Key pair: create a new RSA key pair named jenkins-key and download the .pem file. Store it securely; never upload it to GitHub or share it.
- Network: enable Auto-assign public IP.
- Storage: use the default only if it gives enough room; 20–30 GiB is a practical learning allocation for Jenkins and Docker images.

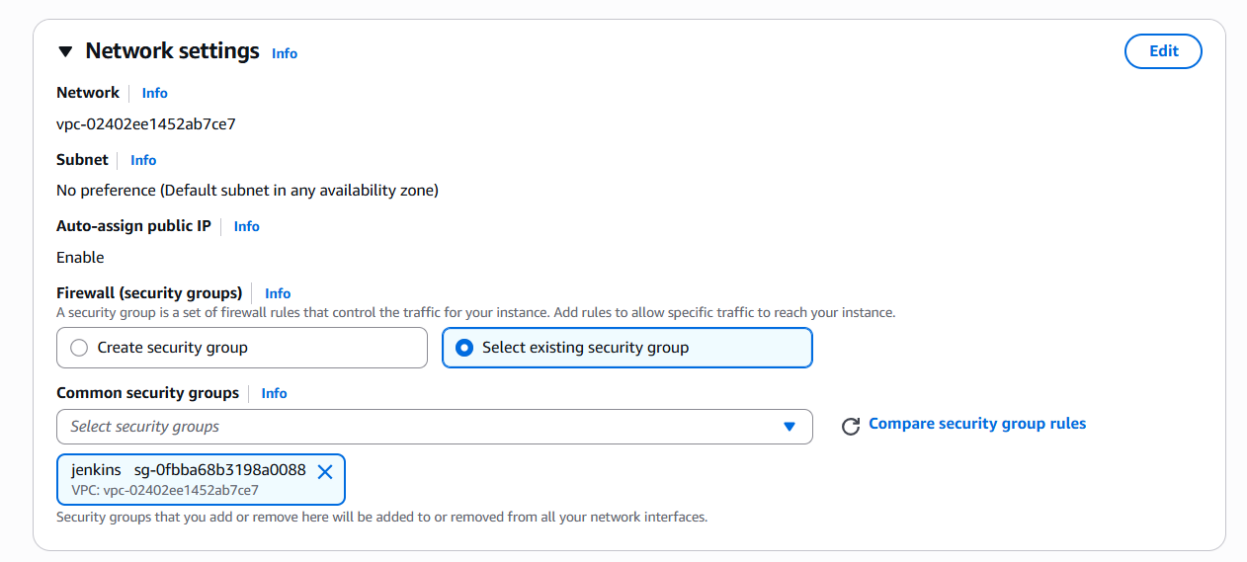


1.3 Security group inbound rules

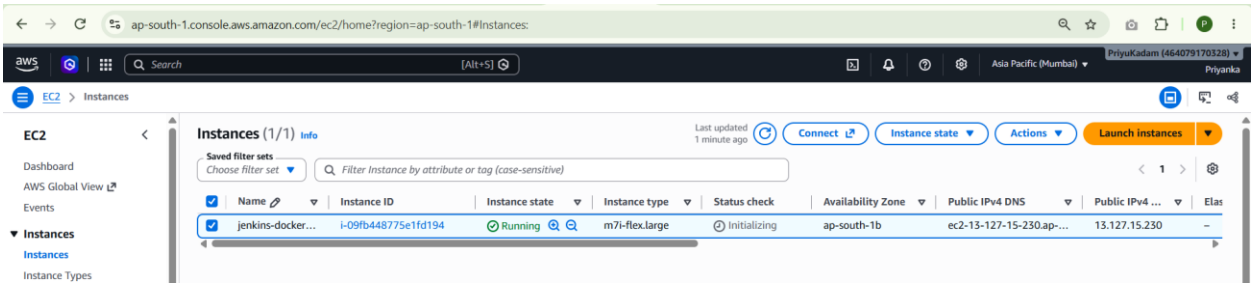
Type / purpose	Port	Source / notes
SSH administration	22	My IP only/Custom
Jenkins UI + webhook	8080	Temporary learning setup: webhook needs to reach Jenkins. If using a public GitHub webhook, My IP alone will not work. Prefer HTTPS/reverse proxy and restrict access for any long-term deployment/ 0.0.0.0/0.
Application HTTP	80	0.0.0.0/0 so visitors can view the demo website

Note: may create a security group for which every time u launch instance for jenkins u can use the existing created security group.





4. Review the configuration and click on the Launch instance.
5. Wait until Instance state is Running and Status checks show 2/2 checks passed.
6. Copy the Public IPv4 address; you will use it as EC2_PUBLIC_IP throughout this guide.



2. Connect to the Ubuntu instance

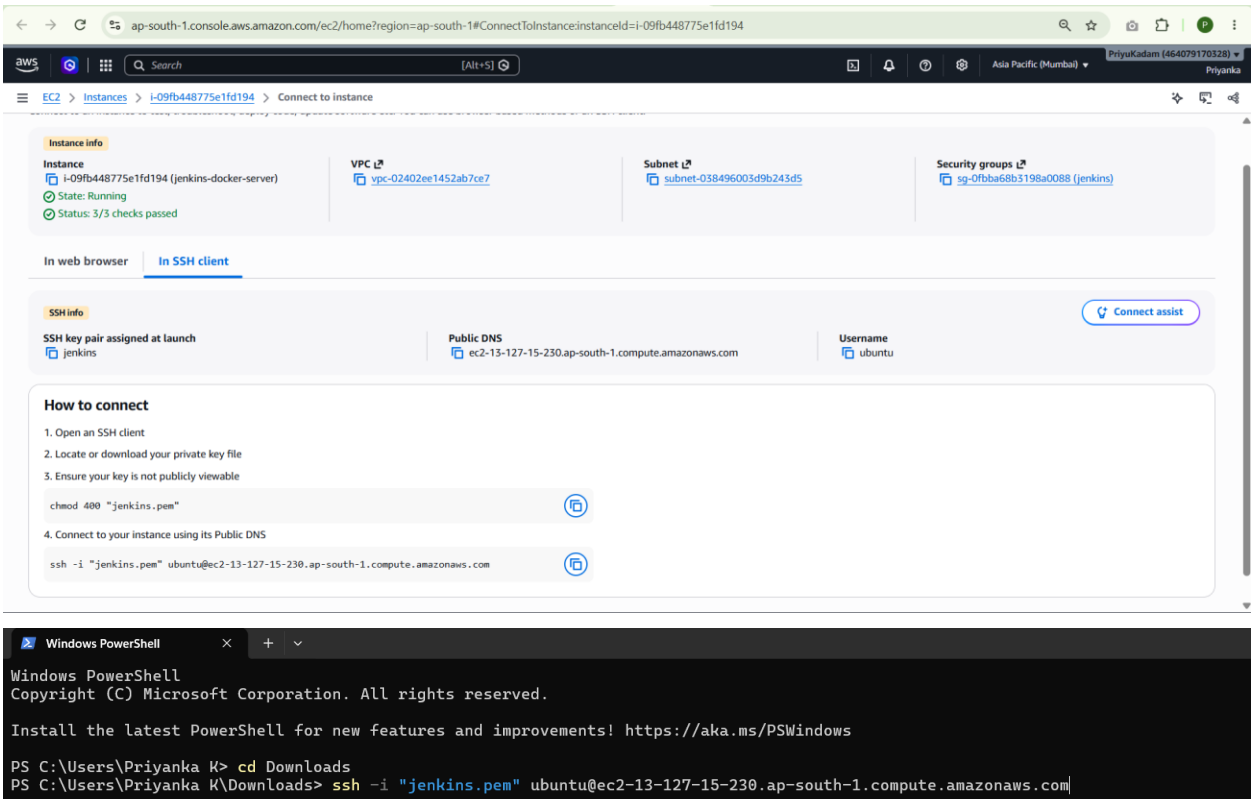
Option A — EC2 Instance Connect

7. In EC2 → Instances, select the instance.
8. Click Connect → EC2 Instance Connect (if available) → Connect.
9. A browser-based terminal should open.

Option B — SSH from Windows PowerShell

10. If needed, move the downloaded jenkins-key.pem file to a safe folder, for example C:\Users\YOUR_USER\.ssh\. Or u can access through download folder itself as well.
11. Open PowerShell and go to that folder.

```
ssh -i "jenkins-key.pem" ubuntu@EC2_PUBLIC_IP
```



Update Ubuntu

```
sudo apt update && sudo apt upgrade -y
```

3. Install Java and Jenkins

Jenkins LTS requires a supported Java runtime. The commands below follow the current official Jenkins LTS apt-repository instructions for Ubuntu/Debian.

3.1 Install Java 21

```
sudo apt update
sudo apt install -y fontconfig openjdk-21-jre
java -version
```

Confirm that the output reports Java 21.

3.2 Check the official Jenkins LTS repository and follow it- https://www.jenkins.io/doc/book/installing/linux/#debianubuntu

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install -y jenkins -y
```

3.3 Start Jenkins and enable it at boot

```
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
```

Look for Active: active (running). Press q to leave the status view.

3.4 Read the initial administrator password

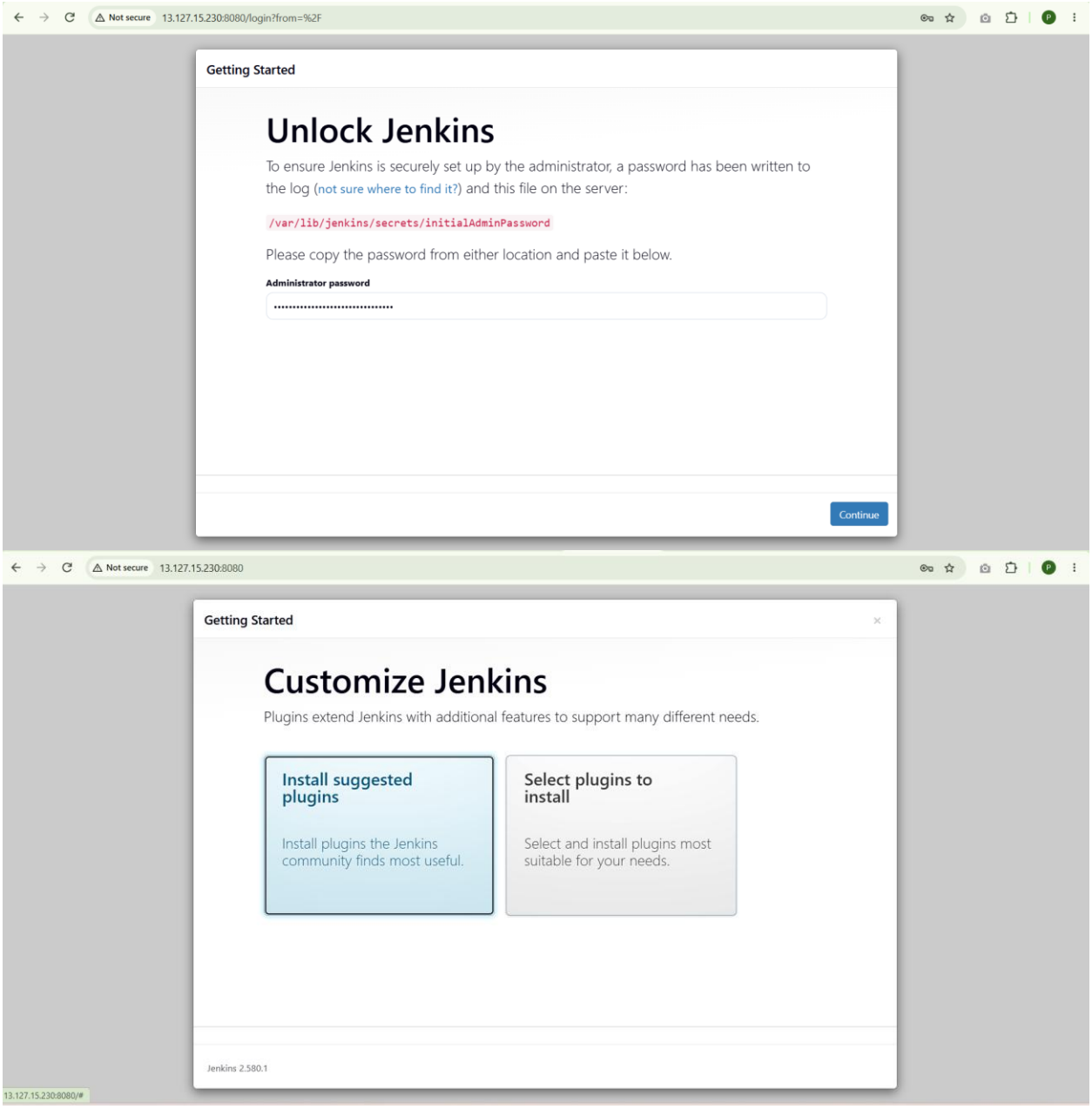
```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

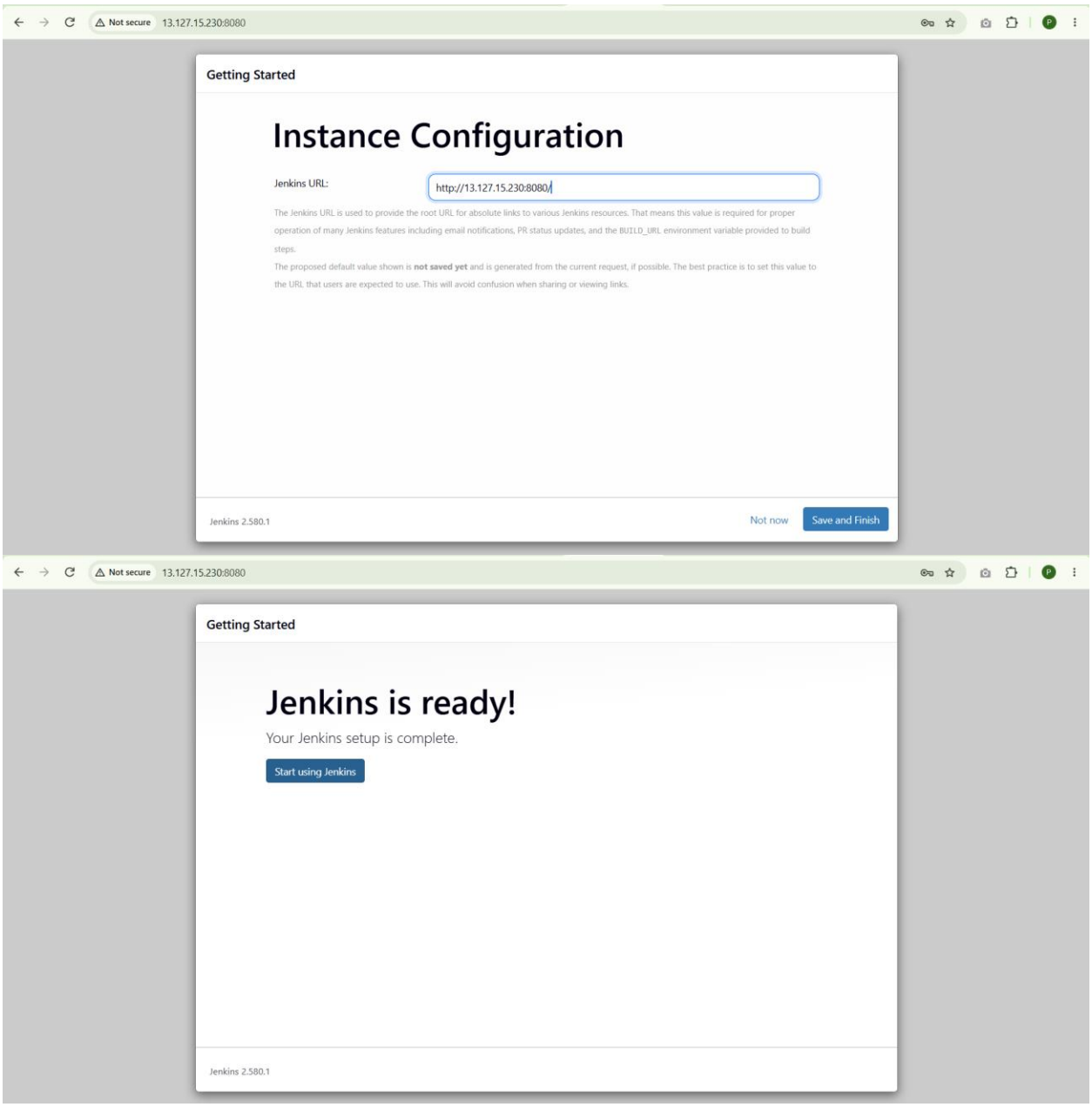
Copy the password temporarily. Do not post it publicly or commit it to a repository.



4. Complete Jenkins first-time setup

12. On your computer, open http://EC2_PUBLIC_IP:8080 in a browser.
13. If the page does not open, confirm Jenkins is running and that the EC2 security group allows inbound TCP 8080 from your current network.
14. Paste the initial administrator password.
15. Choose Install suggested plugins.
16. Create an administrator user with a strong, unique password. Save the credentials safely.
17. Confirm the Jenkins URL and finish setup.


Not secure 13.127.15.230:8080/login?from=%2F
Getting Started
Unlock Jenkins
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:
/var/lib/jenkins/secrets/initialAdminPassword
Please copy the password from either location and paste it below.
Administrator password
Continue
Not secure 13.127.15.230:8080
Getting Started
Customize Jenkins
Plugins extend Jenkins with additional features to support many different needs.
Install suggested plugins
Install plugins the Jenkins community finds most useful.
Select plugins to install
Select and install plugins most suitable for your needs.
Jenkins 2.580.1
13.127.15.230:8080/#



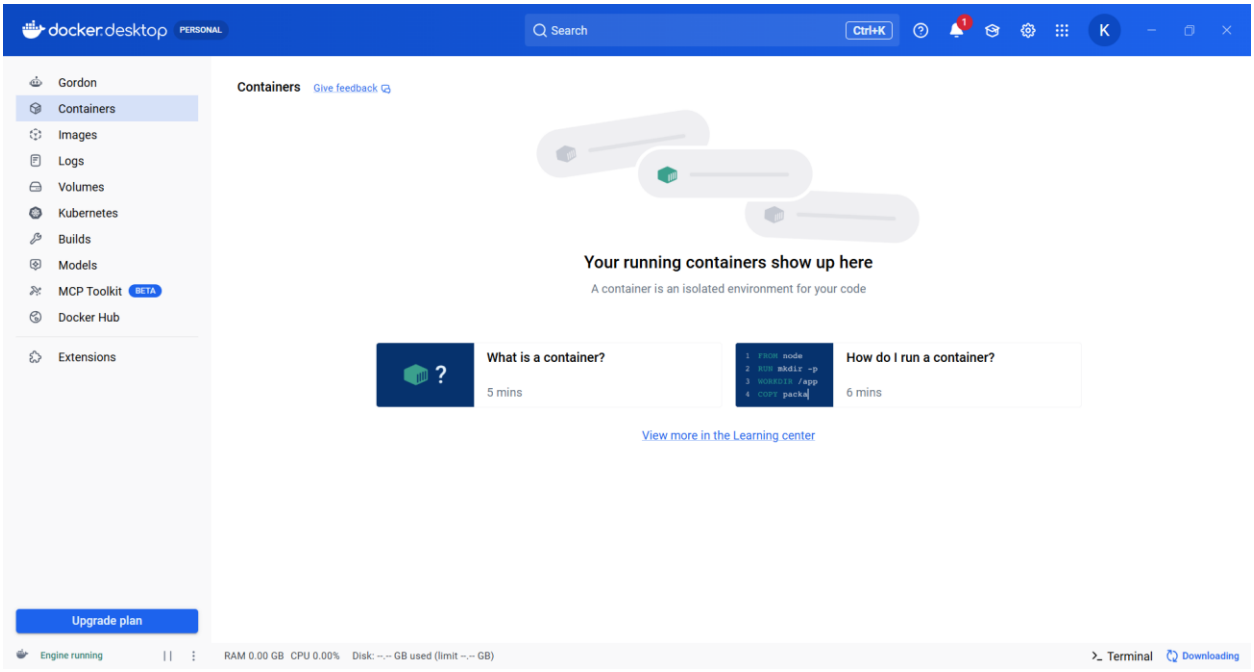
4.1 Install/check required plugins

Go to Manage Jenkins → Plugins. Install or confirm these plugins are available: Git, Pipeline, and GitHub. Depending on the Jenkins version, some may already be installed. Restart Jenkins if prompted.

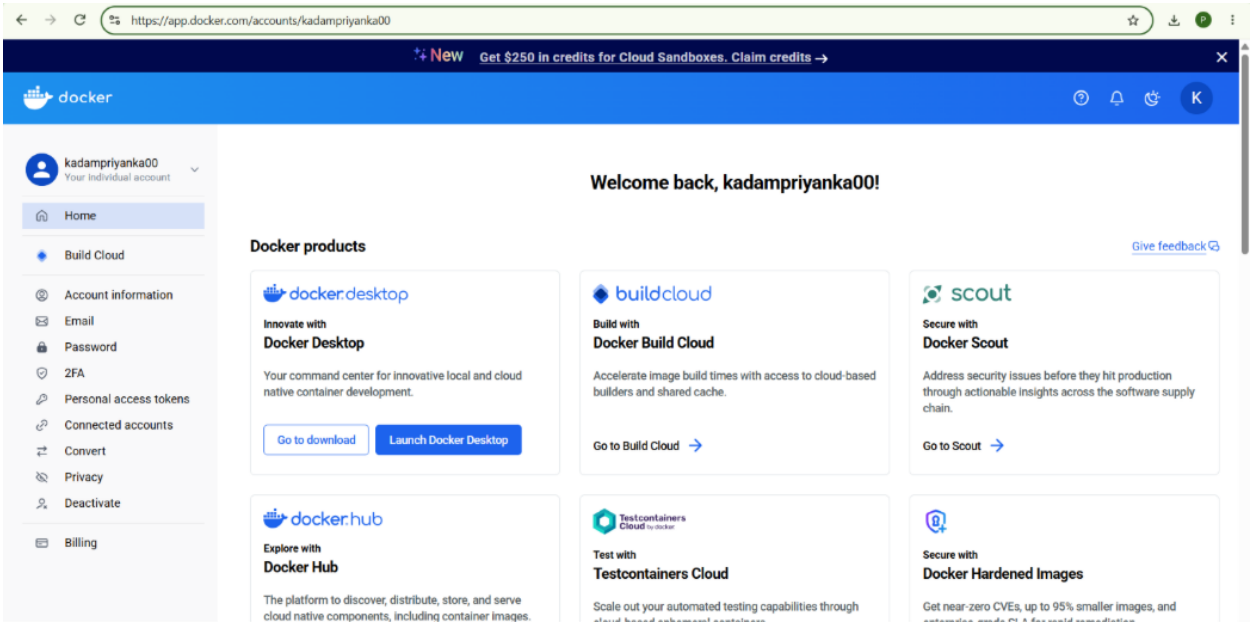
Never disable Jenkins security to make a webhook work. Keep an administrator login enabled.

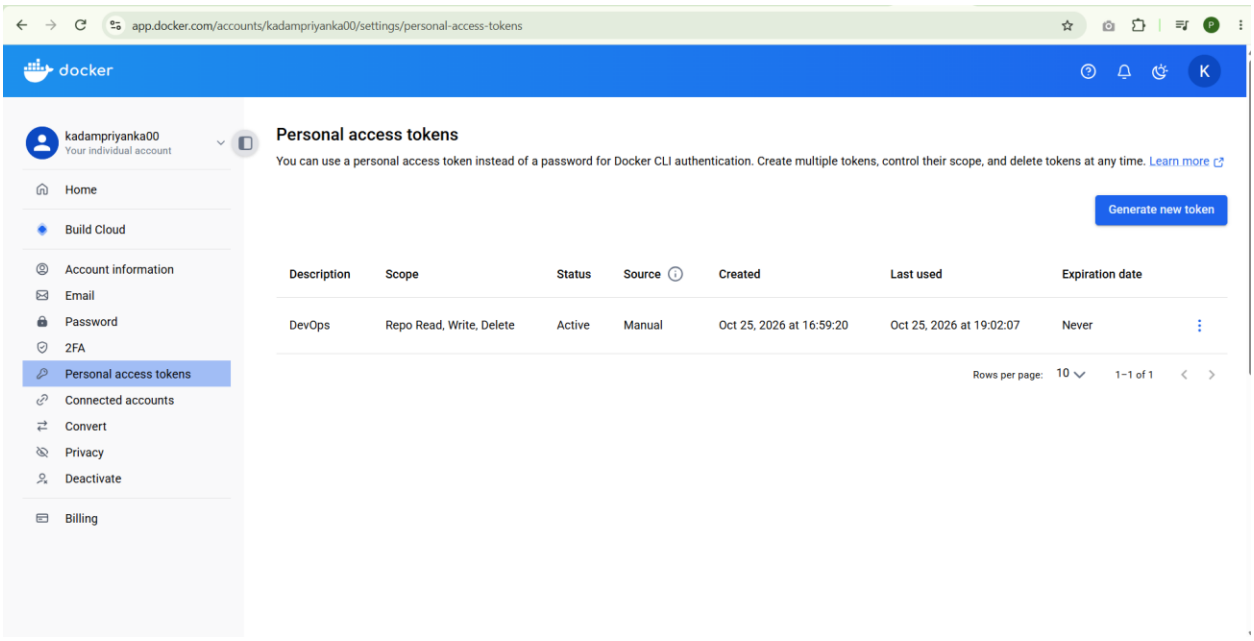
5. Download Docker Desktop and open Docker Hub -

https://docs.docker.com/desktop/setup/install/windows-install/

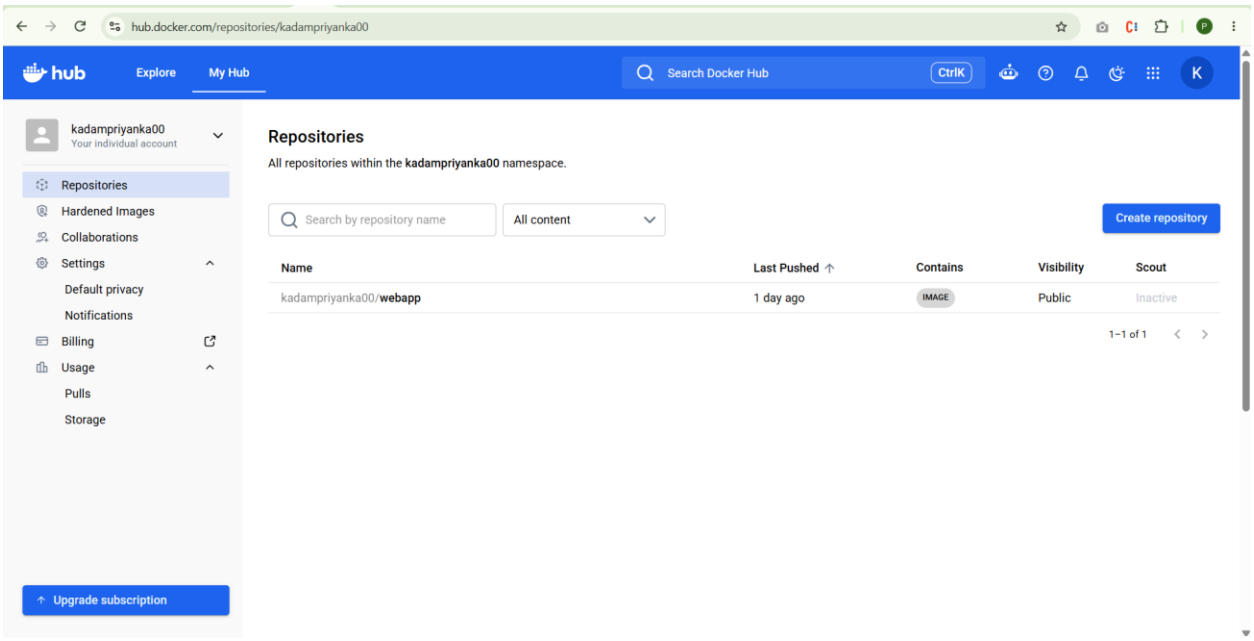


Go to app.docker.com for generating token to put in the jenkin credential.





Create a docker repository -

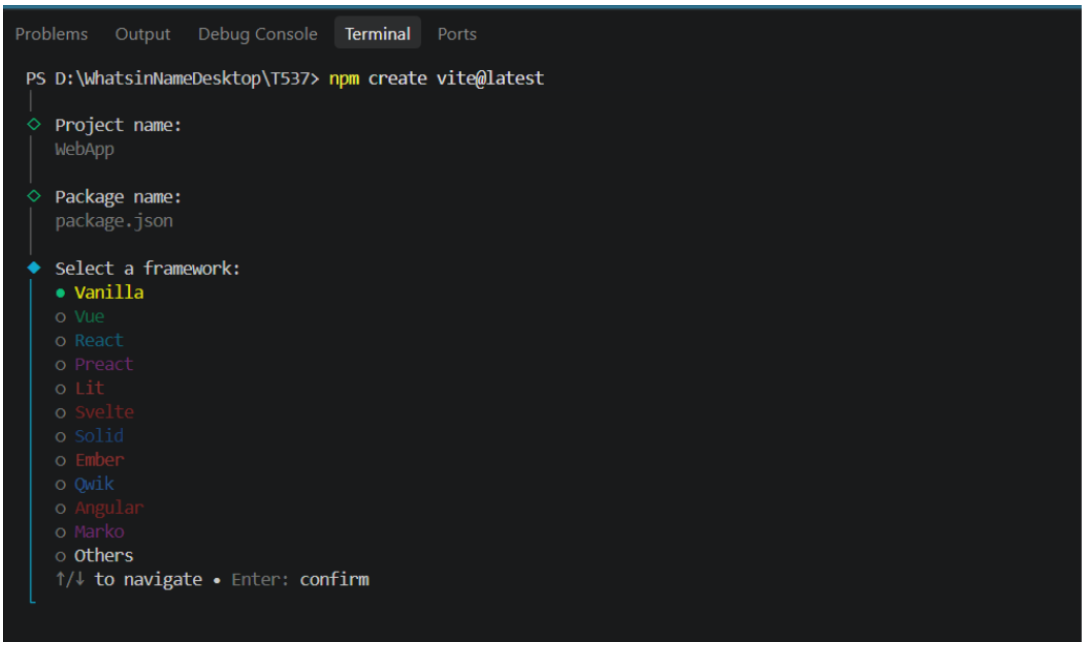


6. Create the GitHub repository

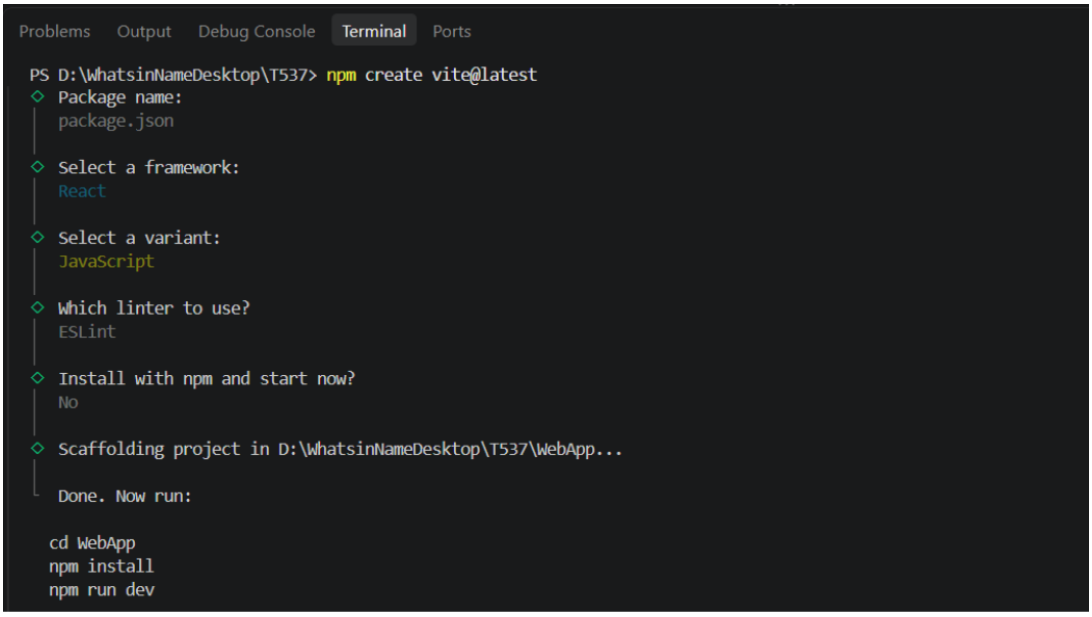
18. Sign in to GitHub and click New repository.
19. Repository name: jenkins-docker-demo.
20. For the simplest learning path, create it as Public. If it is Private, you will need to configure a GitHub credential in Jenkins.
21. Create the repository. You can create files directly in GitHub's browser editor or clone it to your computer.

7. Add the application, Dockerfile, and Jenkinsfile -

7.1 Creating docker file using node -> creating image -> container



linter – auto



(Boilerplate of react code will be available with all of the react folders/files)

7.2 Create Dockerfile

Create a file named Dockerfile (capital D, no file extension):

```
# Docker CLI
FROM node:24 AS build

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

FROM nginx:latest
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD [ "nginx", "-g", "daemon off;" ]
```

This image copies the HTML page into Nginx's default web root. Docker's EXPOSE instruction documents the container port; the pipeline will publish it to host port 80.

7.3 Create Jenkinsfile

Create a file named Jenkinsfile (no extension) in the repository root:

```
pipeline{
    agent any
    environment{
        REPOSITORY = 'webapp'
        IMAGE_NAME = 'webapp'
        IMAGE_TAG = "${BUILD_NUMBER}"
        USERNAME = 'kadampriyanka00'
    }
    stages{
        stage("Checkout"){
            steps{
                checkout scm
            }
        }
        stage("build"){
            steps{
                sh'''
                docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .
                '''
            }
        }
```

```
        stage('DockerHub Login and Push') {
            steps {
                withCredentials([
                    usernamePassword(
                        credentialsId: 'dockerhub-credentials',
                        usernameVariable: 'DOCKERHUB_USERNAME',
                        passwordVariable: 'DOCKERHUB_TOKEN'
                    )
                ]) {
                    sh '''
                        printf '%s' "$DOCKERHUB_TOKEN" | \
                          docker login -u "$DOCKERHUB_USERNAME" --password-stdin

                        docker tag $IMAGE_NAME:$IMAGE_TAG \
                          $DOCKERHUB_USERNAME/$IMAGE_NAME:$IMAGE_TAG

                        docker push \
                          $DOCKERHUB_USERNAME/$IMAGE_NAME:$IMAGE_TAG

                        docker logout
                    '''
                }
            }
        }
        stage("Deploy"){
            steps{
                sh'''
                docker run -d -p 80:80 ${IMAGE_NAME}:${IMAGE_TAG}
                '''
            }
        }
    }
}
```

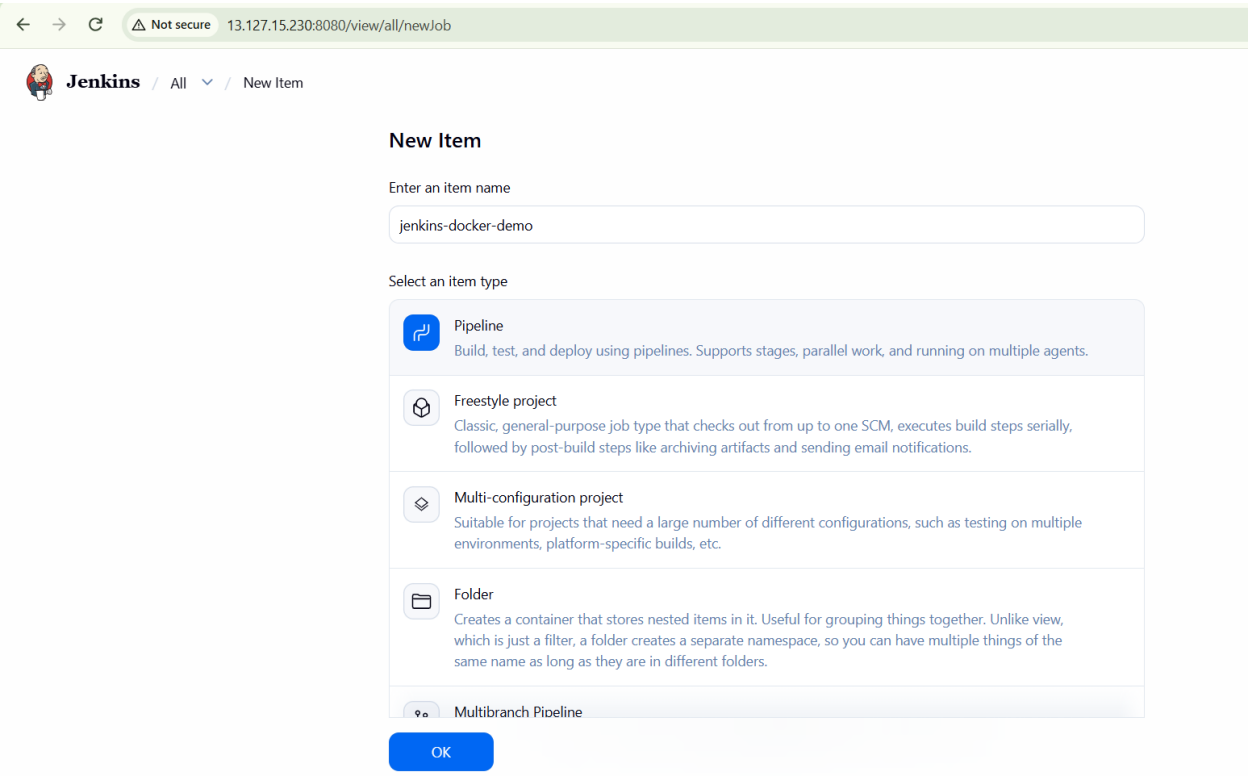
The Jenkins agent runs on the same EC2 host in this learning setup. Port 80 must be free. If another web server already uses port 80, stop it or change the published port and security group accordingly.

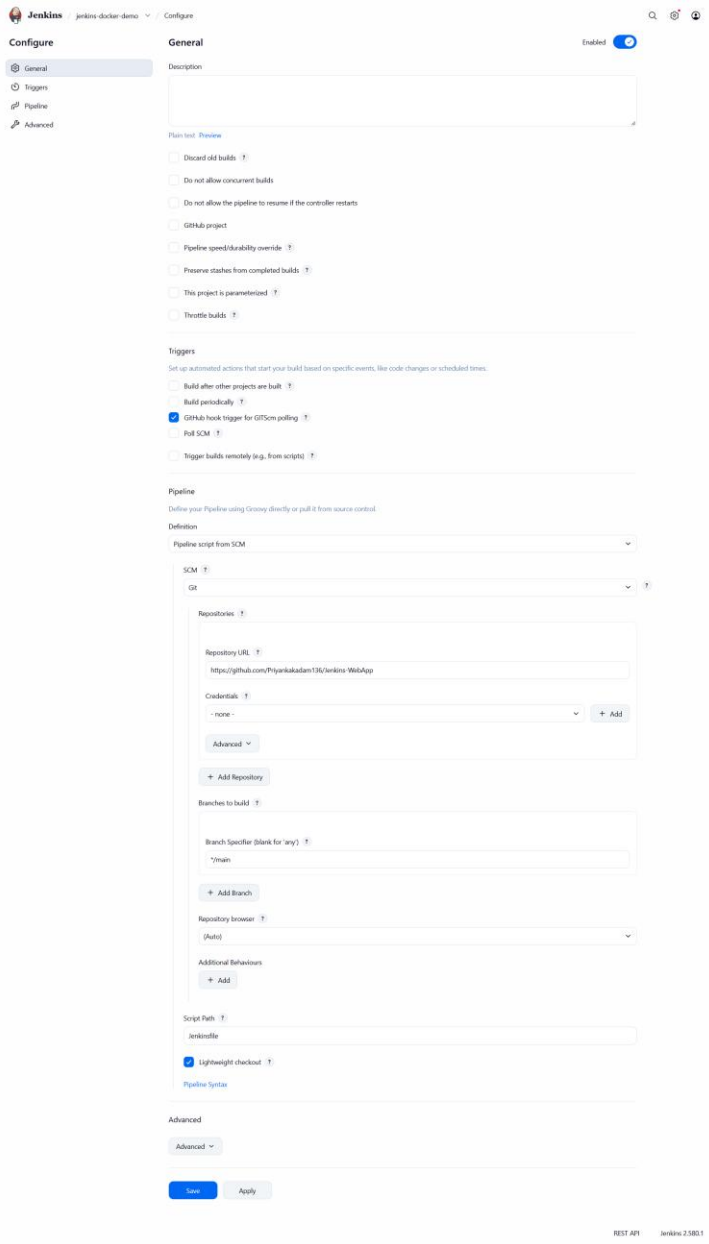
7.4 Commit the files

22. git add . && git commit –m "first commit" && git push origin main
23. Check that all files are visible at the top level of the repository.

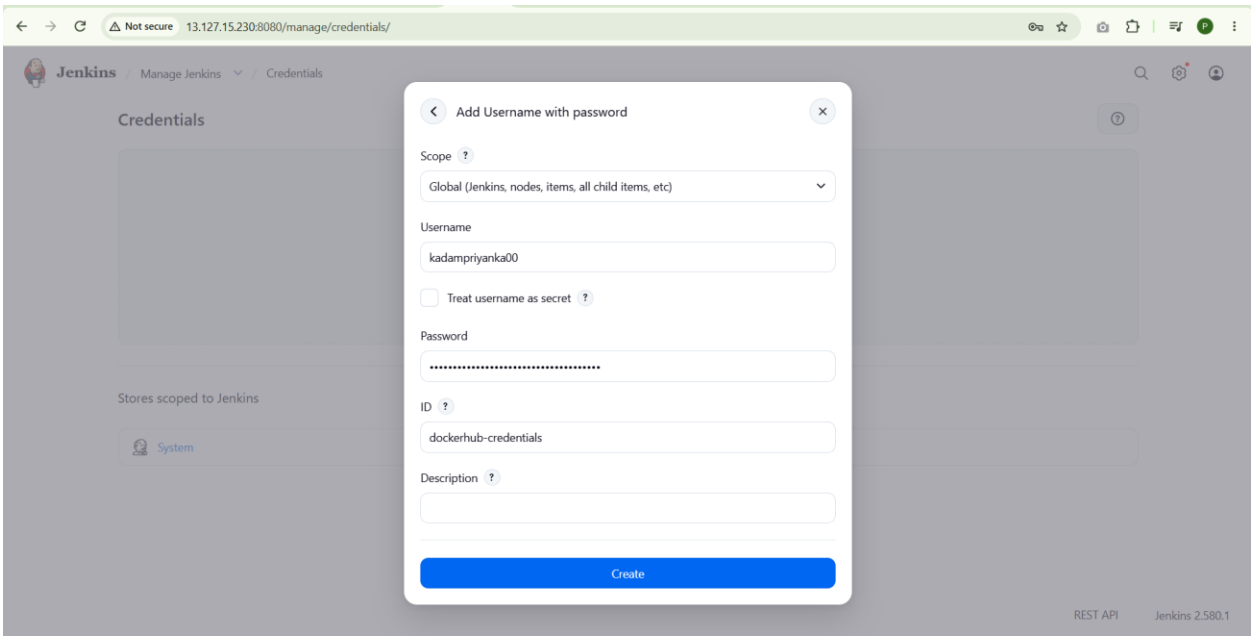
8. Create a Jenkins Pipeline job

24. Open Jenkins at http://EC2_PUBLIC_IP:8080 and sign in.
25. Click New Item.
26. Enter job name: jenkins-docker-demo.
27. Select Pipeline and click OK.
28. Go to configire >> Under the Pipeline section, set Definition to Pipeline script from SCM.
29. SCM: Git.
30. Repository URL: paste the HTTPS URL of your GitHub repository, for example https://github.com/YOUR_USERNAME/jenkins-docker-demo.git.
31. Credentials: leave as none for a public repository. For a private repository, add a suitable GitHub credential and select it; never paste a personal access token into the Jenkinsfile.
32. Branch Specifier: */master (change to */main if that is your actual default branch).
33. Script Path: Jenkinsfile.
34. Under Build Triggers, select GitHub hook trigger for GITScm polling if that option is present.
35. Click Save.




Jenkins
jenkins-docker-demo
Configure
Configure
General
Triggers
Pipeline
Advanced
General
Enabled
Description
Plain text Preview
Discard old builds
Do not allow concurrent builds
Do not allow the pipeline to resume if the controller restarts
GitHub project
Pipeline speed/durability override
Preserve stashes from completed builds
This project is parameterized
Throttle builds
Triggers
Set up automated actions that start your build based on specific events, like code changes or scheduled times.
Build after other projects are built
Build periodically
GitHub hook trigger for GITScm polling
Poll SCM
Trigger builds remotely (e.g., from scripts)
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/Priyankakadam136/Jenkins-WebApp
Credentials
- none -
+ Add
Advanced
+ Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/main
+ Add Branch
Repository browser
(Auto)
Additional Behaviours
+ Add
Script Path
Jenkinsfile
Lightweight checkout
Pipeline Syntax
Advanced
Advanced
Save
Apply
REST API
Jenkins 2.480.1

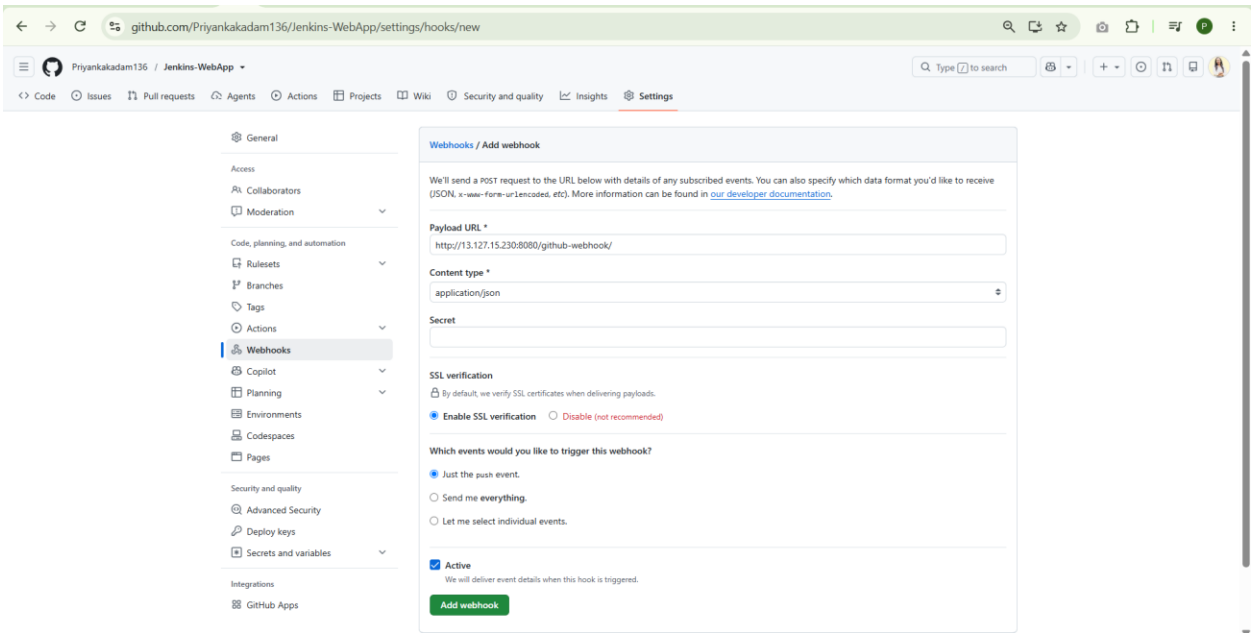
Manage Jenkins >> Credentials >> Add Credential >> Username with password >>(Password(PAT pasword of docker profile)) Create.

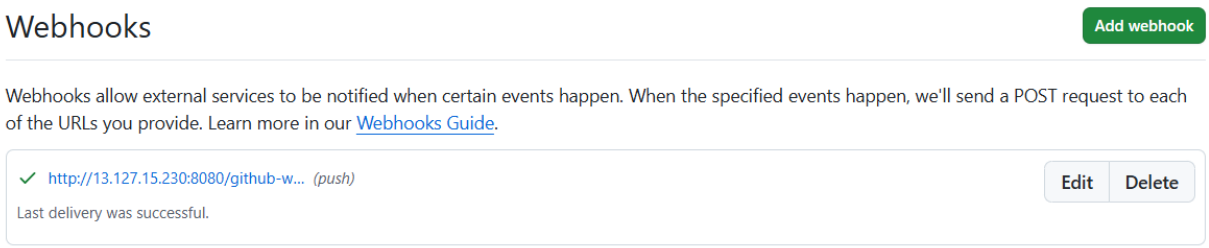


9. Configure the GitHub webhook

The webhook tells Jenkins when a commit is pushed. GitHub must be able to reach the Jenkins webhook URL over the internet.

36. In GitHub, open your repository → Settings → Webhooks → Add webhook.
37. Payload URL: http://EC2_PUBLIC_IP:8080/github-webhook/ (replace EC2_PUBLIC_IP with the current public IPv4 address).
38. Content type: application/json.
39. Secret: for a basic learning setup, you may leave it blank if your Jenkins/plugin configuration does not require one. For a hardened setup, configure webhook-secret validation supported by your integration and use HTTPS.
40. Choose Just the push event.
41. Ensure Active is checked, then click Add webhook.
42. Return to the Webhooks page and inspect Recent Deliveries. A successful delivery normally shows a 2xx response.





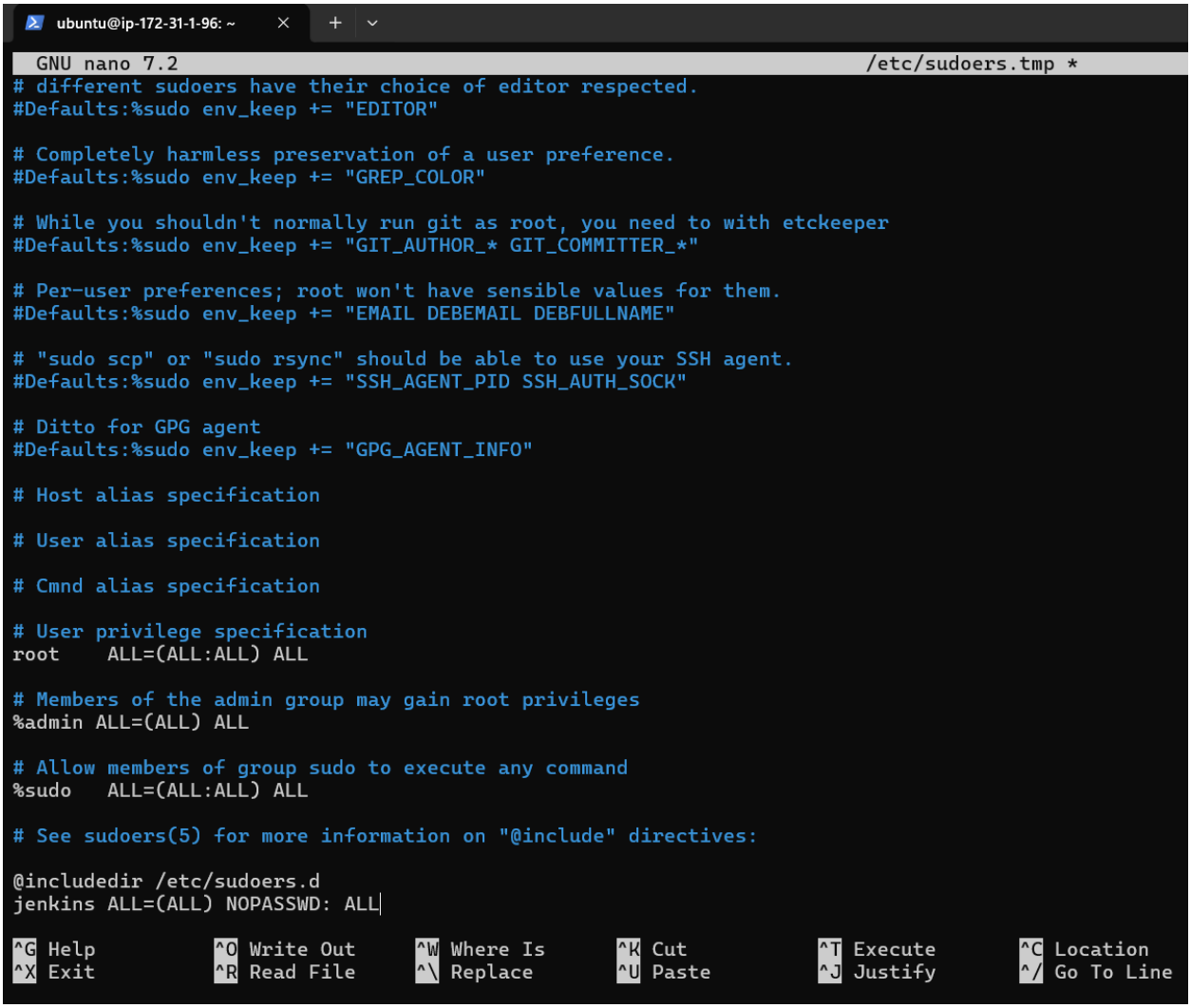
Security warning: exposing Jenkins on plain HTTP port 8080 to the public internet is not appropriate for production. Use HTTPS, a reverse proxy or load balancer, strong authentication, and restricted network access. For a short-lived learning exercise, open 8080 only as long as needed and close it afterward.

Follow the following commands to install docker in jenkins -

```
25  sudo visudo
26  sudo su - jenkins
```

after sudo visudo >> go at last of this file and type jenkins ALL=(ALL) NOPASSWD: ALL

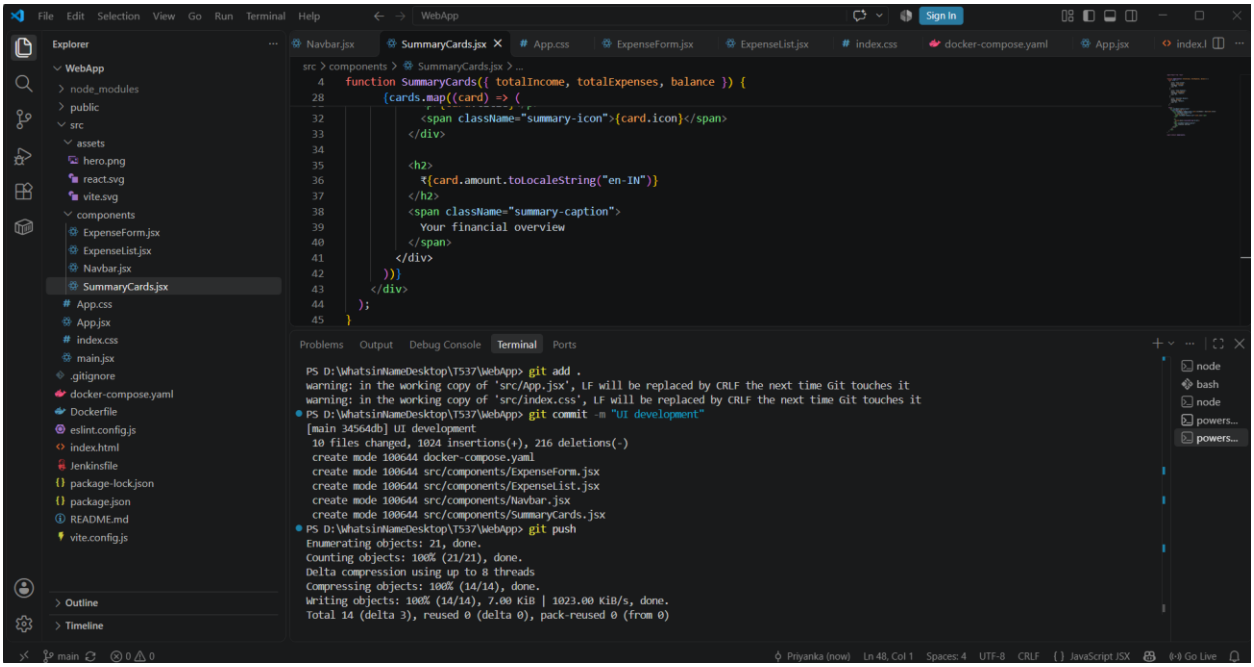
Nano file >> Ctrl + O > Enter > Ctrl + X > Y > Enter



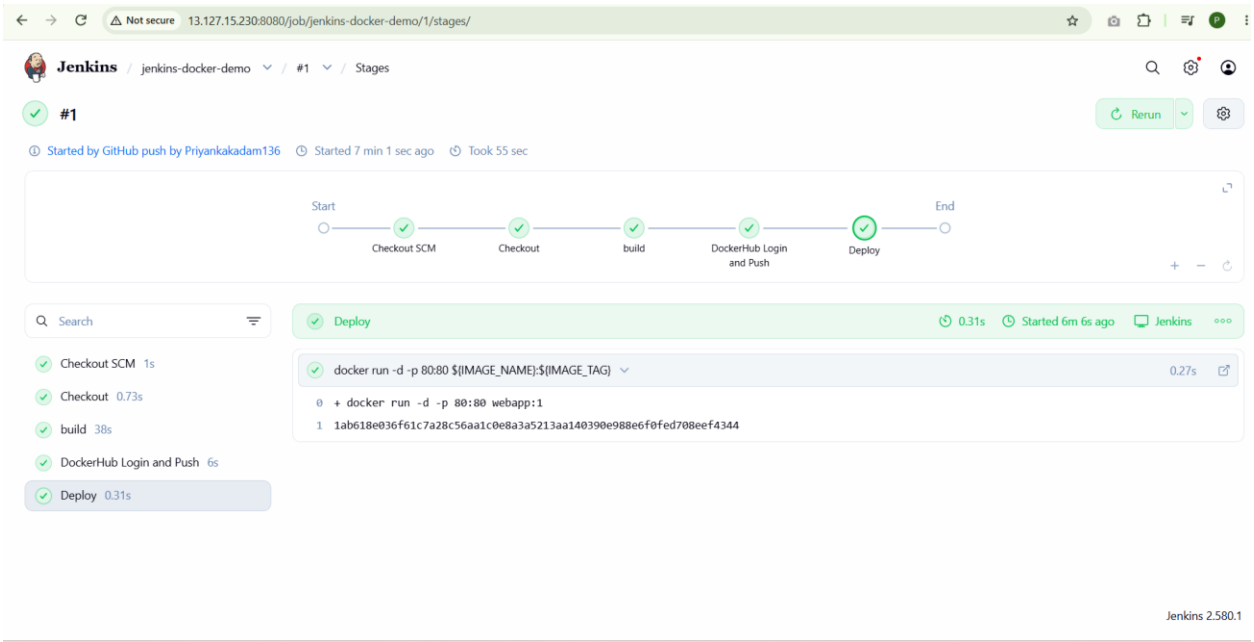
sudo su – jenkins – you go to jenkins terminal to install docker in jenkins -

```
jenkins@ip-172-31-1-96:~$ history
    1  exit
    2  exit
    3  sudo apt install docker.io
    4  sudo chmod 777 /var/run/docker.sock
    5  docker ps
    6  history
```

Update the UI as per your requirement then -
In Git terminal – git add . > git commit –m "updated file" > git push



Successfully jenkins build automatically.



9.1 If GitHub cannot reach Jenkins

- Check that the EC2 instance has a public IPv4 address and is running.
- Check the EC2 security group and any network ACL/firewall rules for inbound TCP 8080.

- A source rule of My IP only will block GitHub's webhook servers. For a temporary test, allow inbound 8080 from the internet only with full awareness of the risk; better is an HTTPS endpoint and carefully restricted access.
- Verify the URL ends with /github-webhook/ and the Jenkins GitHub plugin is installed.
- Check the webhook delivery response and Jenkins logs before retrying.

10. Test the complete CI/CD flow

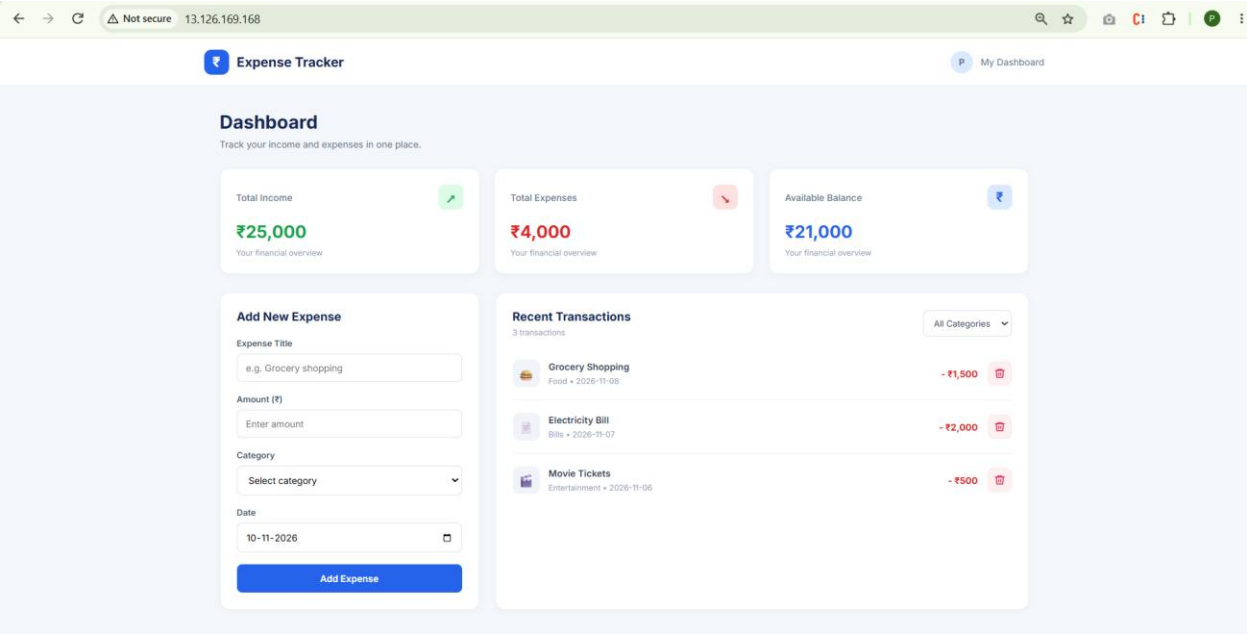
10.1 Push a change

43. Edit index.html in GitHub; change the heading, for example to 'My First Automated Deployment'.
44. Commit the change to main.
45. Open GitHub → Settings → Webhooks → Recent Deliveries and confirm the push delivery succeeded.
46. Open Jenkins and check whether a new build started automatically.
47. Open the build's Console Output and confirm it completes successfully.

10.2 View the website

```
http://EC2_PUBLIC_IP
```

Open the URL in your browser. You should see the updated page. If you use a public IPv4 address, it may change after stopping and starting the instance unless you attach an Elastic IP; Elastic IPs may incur charges depending on use and AWS pricing.



10.3 Verify from the EC2 terminal

```
docker ps
docker images
curl -I http://localhost/
```

Expected: the jenkins-docker-demo container is running, the image exists, and curl reports an HTTP success response.

11. Troubleshooting

Problem	Checks / fix
Jenkins page does not open	Check EC2 state, public IPv4, security-group TCP 8080 rule, and `sudo systemctl status jenkins`.
Jenkins service will not start	Check Java with `java -version`; inspect logs using `sudo journalctl -u jenkins --no-pager -n 100`.
Docker permission denied in Jenkins	Run `sudo usermod -aG docker jenkins`, restart Jenkins, then test `sudo -u jenkins docker ps`. Check Docker is running.
Git checkout fails	Verify repository URL and branch. For private repositories, configure valid credentials in Jenkins.
Jenkinsfile not found	Ensure the file is named exactly `Jenkinsfile`, with no extension, and is at the repository root and on the configured branch.
GitHub webhook fails	Check `/github-webhook/`, plugin installation, Jenkins reachability, and inbound TCP 8080. Review GitHub Recent Deliveries.
Port 80 already in use	Run `sudo ss -ltnp | grep ':80'` and stop/reconfigure the conflicting service, or publish to another port and open that port in the security group.
Website times out	Check that the container is running, host port 80 is published, the security group allows inbound TCP 80, and the public IP is correct.
Build succeeds but old page appears	Confirm the latest commit is on the configured branch and the new image was built. Hard-refresh the browser and inspect `docker logs jenkins-docker-demo`.
Instance runs out of memory	Use a larger instance, reduce concurrent builds, and avoid running unnecessary containers. Check `free -h` and `df -h`.

12. Security and cost cleanup

Security checklist

- Never commit AWS access keys, private SSH keys, Jenkins passwords, or GitHub tokens to GitHub.
- Use a strong Jenkins admin password and keep Jenkins security enabled.
- Do not leave port 8080 open to the entire internet longer than needed. Prefer HTTPS and restricted access.
- Keep SSH port 22 restricted to your IP or a trusted network.
- Remember: a user in the Docker group can effectively control the host as root.
- Use IAM roles and least-privilege permissions if the project later needs access to AWS APIs.
- This single-instance architecture is for learning. For production, separate CI from application hosting and use a secured deployment design.

Avoid unexpected AWS charges

- Check the current EC2, EBS storage, data transfer, and Elastic IP pricing in your region.
- Stopping an EC2 instance usually stops its compute charges, but attached EBS storage can still incur charges.

- To stop charges for this practice environment, back up anything needed, then terminate the instance and remove unused resources you created, such as unattached volumes or Elastic IPs. Termination is destructive.
- Do not terminate the instance until you have saved any project files or notes you need.

Official documentation

- **Amazon EC2 — Launch an instance:** https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/LaunchingAndUsingInstances.html
- **Amazon EC2 — Security groups:** https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html
- **Jenkins — Installing on Linux:** https://www.jenkins.io/doc/book/installing/linux/
- **Jenkins — GitHub plugin:** https://plugins.jenkins.io/github/
- **Jenkins — Pipeline syntax:** https://www.jenkins.io/doc/book/pipeline/syntax/
- **Docker Engine — Install on Ubuntu:** https://docs.docker.com/engine/install/ubuntu/
- **GitHub — Creating webhooks:** https://docs.github.com/en/webhooks/using-webhooks/creating-webhooks

Completion target: a commit to GitHub triggers Jenkins automatically, Jenkins builds a Docker image and replaces the running container, and the updated application is visible at http://EC2_PUBLIC_IP.